下面直接给你一套可落地的 **Disease Backbone Builder** 设计，包括：

1. 完整 schema
2. LLM 抽取 prompt
3. 聚合规则
4. backbone 生成流水线

这套设计的核心，严格对应 disease map 论文里的几个原则：先定义 disease scope 和 biological coverage，再把知识 formalise 成标准化、分层、模块化、可证据追踪的表示。论文还强调模块化、多层结构、interaction/evidence annotation 和后续可视化/分析用途。

一、Disease Backbone Builder 的定位

先给它一个清晰定义：

> **Disease Backbone Builder = 一个把权威疾病资料转成“初步 disease mechanism backbone” 的自动化构建器。**

它的输出不是最终全量 disease map，而是一个 **draft backbone**，包含：

- disease hallmarks
- core mechanism modules
- module relations
- canonical causal chains
- key genes
- source support

后续再由 Map Builder 用 Reactome / GO / STRING / literature / assay evidence 去扩图。

二、整体架构

2.1 顶层流程

```
Input disease
  ↓
Source collection
  ↓
Source packet normalization
  ↓
LLM constrained extraction
  ↓
Candidate aggregation
  ↓
Normalization and ontology mapping
  ↓
```

```
Scoring and pruning
  ↓
Backbone assembly
  ↓
Validation
  ↓
Export disease_backbone_draft.json
```

2.2 输入输出

输入

- disease_name
- disease_ids（MONDO / Orphanet / MeSH 等）
- optional seed genes
- source packets

输出

- disease_backbone_draft.json
- extraction artifacts.jsonl
- aggregation summary.json
- review report.md/json

三、完整 schema 设计

我建议分成 10 个核心对象：

1. BuilderConfig
2. DiseaseDescriptor
3. SourcePacket
4. ExtractionResult
5. HallmarkCandidate
6. ModuleCandidate
7. CausalChainCandidate
8. BackboneAggregationRecord
9. DiseaseBackboneDraft
10. ValidationReport

3.1 BuilderConfig schema

```
{
  "builder_id": "disease_backbone_builder_v1",
  "version": "1.0.0",
```

```
  "disease": {
    "label": "Parkinson disease",
    "mondo_id": "MONDO:0005180",
    "mesh_id": "D010300",
    "orphanet_id": null
  },
  "source_policy": {
    "preferred_source_types": [
      "GeneReviews",
      "Orphanet",
      "ReviewArticle",
      "ReactomeSummary",
      "GOSummary"
    ],
    "max_packets_per_source": 20,
    "max_text_chars_per_packet": 12000
  },
  "llm_policy": {
    "provider": "qwen",
    "mode": "json_constrained_extraction",
    "temperature": 0.1,
    "allow_world_knowledge": false,
    "require_source_grounding": true
  },
  "aggregation_policy": {
    "min_support_for_core_module": 2,
    "min_support_for_core_hallmark": 2,
    "min_chain_confidence": 0.7,
    "generic_term_filter_enabled": true
  },
  "output_policy": {
    "emit_review_flags": true,
    "emit_provisional_items": true,
    "formats": ["json", "jsonl", "csv"]
  }
}
```

3.2 DiseaseDescriptor schema

```
{
  "label": "Parkinson disease",
  "synonyms": [
    "Parkinson's disease",
    "PD"
  ],
  "ids": {
    "mondo": "MONDO:0005180",
    "mesh": "D010300",
    "orphanet": null,
```

```
    "omim": null
  },
  "seed_genes": [
    "PRKN",
    "PINK1",
    "LRRK2",
    "GBA",
    "SNCA",
    "VPS35"
  ],
  "disease_scope_note": "Build a mechanistic backbone for disease-level
causal interpretation, not full phenome coverage."
}
```

3.3 SourcePacket schema

每个来源先被拆成统一 packet，这是后续一切抽取的入口。

```
{
  "source_packet_id": "sp_gr_pd_001",
  "disease_label": "Parkinson disease",
  "source_type": "GeneReviews",
  "source_name": "GeneReviews",
  "source_title": "Parkinson Disease Overview",
  "source_locator": {
    "url": null,
    "pmid": null,
    "doi": null,
    "internal_ref": "GeneReviews_PD_Overview_section_1"
  },
  "section_label": "Disease mechanism / pathogenesis",
  "text_block": "....",
  "metadata": {
    "language": "en",
    "is_review_like": true,
    "is_authoritative_summary": true,
    "publication_year": null
  }
}
```

source_type 受控词表

```
GeneReviews
Orphanet
ReviewArticle
ConsensusStatement
ReactomeSummary
```

```
GOSummary
ClinGenSummary
OMIMSummary
Other
```

3.4 ExtractionResult schema

这是单个 source packet 经过 LLM 后的结构化输出。

```
{
  "source_packet_id": "sp_gr_pd_001",
  "disease": {
    "label": "Parkinson disease",
    "mondo_id": "MONDO:0005180"
  },
  "hallmarks": [],
  "modules": [],
  "module_relations": [],
  "causal_chains": [],
  "key_genes": [],
  "global_notes": [],
  "extraction_quality": {
    "llm_confidence": 0.89,
    "needs_manual_review": false,
    "warnings": []
  }
}
```

3.5 HallmarkCandidate schema

```
{
  "candidate_id": "hallmark_cand_001",
  "label": "mitochondrial dysfunction",
  "normalized_label": "mitochondrial dysfunction",
  "description": "Mitochondrial homeostasis impairment is a central
pathogenic theme.",
  "evidence_scope": "disease_level",
  "supporting_source_packet_ids": [
    "sp_gr_pd_001",
    "sp_review_pd_003"
  ],
  "supporting_spans": [
    {
      "source_packet_id": "sp_gr_pd_001",
      "quote": "..."
    }
```

```
  ],
  "candidate_confidence": 0.93,
  "status": "candidate"
}
```

evidence_scope

```
disease_level
module_level
process_level
```

3.6 ModuleCandidate schema

```
{
  "candidate_id": "module_cand_001",
  "label": "PINK1-PRKN mediated mitophagy",
  "normalized_label": "mitophagy",
  "description": "Damage sensing and selective removal of impaired
mitochondria.",
  "module_type": "core_mechanism_module",
  "hallmark_links": [
    "mitochondrial dysfunction",
    "impaired mitophagy and autophagy"
  ],
  "key_genes": [
    "PINK1",
    "PRKN",
    "FBXO7"
  ],
  "process_terms": [
    "mitochondrial depolarization sensing",
    "PRKN recruitment",
    "mitochondrial protein ubiquitination",
    "mitophagy initiation"
  ],
  "supporting_source_packet_ids": [
    "sp_gr_pd_001",
    "sp_orpha_pd_001"
  ],
  "candidate_confidence": 0.92,
  "status": "candidate"
}
```

module_type

```
core_mechanism_module
supporting_module
phenotype_convergence_module
peripheral_module
```

3.7 ModuleRelation schema

```
{
  "candidate_id": "module_rel_001",
  "subject_module": "mitochondrial quality control",
  "predicate": "upstream_of",
  "object_module": "mitophagy",
  "description": "Mitochondrial damage sensing is upstream of selective
mitochondrial clearance.",
  "supporting_source_packet_ids": [
    "sp_gr_pd_001"
  ],
  "candidate_confidence": 0.82
}
```

predicate 受控词表

```
upstream_of
downstream_of
interacts_with
converges_on
amplifies
impairs
supports
linked_to
```

3.8 CausalChainCandidate schema

```
{
  "candidate_id": "chain_cand_001",
  "title": "Mitophagy failure drives neuronal degeneration",
  "module_label": "mitophagy",
  "steps": [
    {
      "order": 1,
      "event_label": "mitochondrial depolarization"
    },
```

```
    {
      "order": 2,
      "event_label": "PINK1 stabilization on damaged mitochondria"
    },
    {
      "order": 3,
      "event_label": "PRKN recruitment"
    },
    {
      "order": 4,
      "event_label": "mitochondrial protein ubiquitination"
    },
    {
      "order": 5,
      "event_label": "mitophagy initiation"
    },
    {
      "order": 6,
      "event_label": "damaged mitochondria clearance"
    },
    {
      "order": 7,
      "event_label": "dopaminergic neuron protection"
    }
  ],
  "trigger_examples": [
    "PINK1 loss",
    "PRKN loss"
  ],
  "supporting_source_packet_ids": [
    "sp_gr_pd_001",
    "sp_review_pd_003"
  ],
  "candidate_confidence": 0.9,
  "status": "candidate"
}
```

3.9 KeyGeneCandidate schema

```
{
  "candidate_id": "gene_cand_001",
  "symbol": "PRKN",
  "gene_role": "core_driver",
  "linked_modules": [
    "mitophagy",
    "mitochondrial quality control"
  ],
  "rationale": "Repeatedly implicated as a direct mediator of damaged
mitochondria clearance.",
```

```
  "supporting_source_packet_ids": [
    "sp_gr_pd_001",
    "sp_orpha_pd_001"
  ],
  "candidate_confidence": 0.95
}
```

gene_role

```
core_driver
major_associated_gene
module_specific_gene
supporting_gene
uncertain
```

3.10 BackboneAggregationRecord schema

这是聚合时的中间层。

```
{
  "aggregation_id": "agg_mod_001",
  "item_type": "module",
  "normalized_key": "mitophagy",
  "merged_labels": [
    "PINK1-PRKN mediated mitophagy",
    "mitophagy",
    "selective mitochondrial autophagy"
  ],
  "source_count": 3,
  "source_packet_ids": [
    "sp_gr_pd_001",
    "sp_orpha_pd_001",
    "sp_review_pd_003"
  ],
  "merged_key_genes": [
    "PINK1",
    "PRKN",
    "FBXO7"
  ],
  "merged_process_terms": [
    "PINK1 stabilization",
    "PRKN recruitment",
    "mitophagy initiation"
  ],
  "support_score": 0.91,
  "review_flags": []
}
```

3.11 DiseaseBackboneDraft schema

这是最终产物。

```
{
  "backbone_id": "pd_backbone_draft_v1",
  "builder_version": "1.0.0",
  "disease": {
    "label": "Parkinson disease",
    "ids": {
      "mondo": "MONDO:0005180",
      "mesh": "D010300"
    }
  },
  "hallmarks": [],
  "modules": [],
  "module_relations": [],
  "canonical_chains": [],
  "key_genes": [],
  "source_summary": {
    "source_packet_count": 8,
    "source_type_counts": {
      "GeneReviews": 2,
      "Orphanet": 1,
      "ReviewArticle": 5
    }
  },
  "build_quality": {
    "overall_confidence": 0.88,
    "items_needing_review": 3,
    "provisional_item_count": 5
  },
  "status": "draft"
}
```

3.12 ValidationReport schema

```
{
  "backbone_id": "pd_backbone_draft_v1",
  "validation_passed": true,
  "checks": {
    "hallmark_count_ok": true,
    "module_count_ok": true,
    "all_core_modules_have_support": true,
    "all_chains_have_multiple_steps": true,
```

```
    "all_key_genes_linked_to_module": true,
    "generic_module_filter_passed": true
  },
  "warnings": [
    "Module 'cell stress response' is too generic and remains
provisional."
  ],
  "review_recommendations": [
    "Review relationship between lysosomal dysfunction and alpha-
synuclein aggregation."
  ]
}
```

四、LLM 抽取 prompt 设计

这里最关键的不是写得花，而是 **强约束 + 禁止常识补全 + 只基于输入抽取**。

我建议拆成 3 类 prompt：

1. source packet backbone extraction
2. module refinement extraction
3. cross-source merge assistance

4.1 Prompt A：单个 source packet 抽 backbone 候选

system prompt

```
You are a disease mechanism backbone extractor.

Your task is NOT to write a free-form summary.
Your task is to convert the input source packet into a structured draft
of disease backbone candidates.

You must follow these rules:
1. Use ONLY information explicitly supported by the input text.
2. Do NOT add external knowledge.
3. Do NOT guess missing mechanisms.
4. Prefer disease-relevant mechanisms over general cell biology.
5. Output JSON only.
6. If a mechanism is too generic or weakly supported, mark it as
provisional or omit it.
7. Hallmarks must be disease-level themes.
8. Modules must be mechanism blocks that could be independently expanded
later.
9. Causal chains must be short ordered event sequences, not narrative
paragraphs.
10. Every extracted item must retain source support.
```

user prompt 模板

```
Disease:
{disease_name}

Disease IDs:
{disease_ids_json}

Optional seed genes:
{seed_genes_json}

Source packet metadata:
{source_packet_metadata_json}

Source text:
"""
{source_text}
"""

Return a JSON object with the following fields:

{
  "hallmarks": [
    {
      "label": "...",
      "description": "...",
      "confidence": 0-1,
      "status": "candidate|provisional"
    }
  ],
  "modules": [
    {
      "label": "...",
      "description": "...",
      "module_type":
"core_mechanism_module|supporting_module|phenotype_convergence_module|pe
ripheral_module",
      "hallmark_links": ["..."],
      "key_genes": ["..."],
      "process_terms": ["..."],
      "confidence": 0-1,
      "status": "candidate|provisional"
    }
  ],
  "module_relations": [
    {
      "subject_module": "...",
      "predicate":
"upstream_of|downstream_of|interacts_with|converges_on|amplifies|impairs
|supports|linked_to",
```

```
      "object_module": "...",
      "description": "...",
      "confidence": 0-1
    }
  ],
  "causal_chains": [
    {
      "title": "...",
      "module_label": "...",
      "steps": [
        {"order": 1, "event_label": "..."},
        {"order": 2, "event_label": "..."}
      ],
      "trigger_examples": ["..."],
      "confidence": 0-1,
      "status": "candidate|provisional"
    }
  ],
  "key_genes": [
    {
      "symbol": "...",
      "gene_role":
"core_driver|major_associated_gene|module_specific_gene|supporting_gene|
uncertain",
      "linked_modules": ["..."],
      "rationale": "...",
      "confidence": 0-1
    }
  ],
  "global_notes": ["..."]
}

Additional constraints:
- Extract 3-8 hallmarks at most.
- Extract 3-12 modules at most.
- Prefer modules that are mechanistically meaningful for disease
interpretation.
- Avoid generic labels such as "cell stress", "metabolism",
"inflammation" unless the text makes them specifically disease-central.
- If the text only weakly supports an item, mark it provisional.
```

4.2 Prompt B：模块细化 prompt

当聚合后某个模块被识别出来，可以再细化。

system prompt

```
You are a mechanism module refiner.
```

```
Your task is to refine one disease mechanism module using only the
provided supporting text snippets.
Do not add external knowledge.
Return JSON only.
```

user prompt 模板

```
Disease: {disease_name}
Target module: {module_label}

Supporting snippets:
{snippets_json}

Return:
{
  "normalized_label": "...",
  "definition": "...",
  "key_genes": ["..."],
  "core_process_terms": ["..."],
  "entry_events": ["..."],
  "exit_events": ["..."],
  "canonical_chain": {
    "steps": [
      {"order": 1, "event_label": "..."},
      {"order": 2, "event_label": "..."}
    ]
  },
  "confidence": 0-1,
  "review_flags": ["..."]
}
```

4.3 Prompt C：跨来源合并辅助 prompt

这个 prompt 只在“是否应合并某些近义模块”时使用，不能让 LLM 主导最终 merge。

user prompt 模板

```
Disease: {disease_name}

Candidate modules:
{candidate_modules_json}

Task:
Identify which module labels likely refer to the same mechanism concept.
Use only semantic equivalence, not broad biological relatedness.

Return:
```

```
{
  "merge_groups": [
    {
      "canonical_label": "...",
      "members": ["...", "..."],
      "reason": "..."
    }
  ],
  "do_not_merge": [
    {
      "label_a": "...",
      "label_b": "...",
      "reason": "..."
    }
  ]
}
```

五、聚合规则

这部分决定 backbone 质量，不能只靠 LLM。

5.1 聚合目标

聚合要完成四件事：

1. 合并同义 hallmark/module
2. 合并相近 causal chain
3. 统计支持度
4. 过滤泛化、外围、低支持项

5.2 hallmark 聚合规则

规则

- label 归一化：小写、去标点、词形还原
- 同义短语归并：如
  - mitochondrial dysfunction
  - mitochondrial impairment
  - mitochondrial homeostasis defect

升为 core hallmark 条件

- 至少 2 个独立 source packets 支持
- 或 1 个 GeneReviews + 1 个 high-quality review 支持

降为 provisional 条件

- 仅 1 个来源提及
- 过于泛化
- 与 disease specificity 连接不清

5.3 module 聚合规则

规则 1：模块命名标准化

优先用简洁 canonical label，例如：

- mitophagy
- lysosomal dysfunction
- alpha-synuclein proteostasis
- endolysosomal trafficking
- synaptic vesicle dysfunction

而不是长句式标题。

规则 2：模块可独立扩展

模块必须满足至少两条：

- 有明确过程边界
- 有一组相关 key genes
- 有可能被扩成 process-level subgraph
- 与 disease hallmark 有连接

规则 3：core module 支持阈值

推荐：

- source_count ≥ 2
- candidate_confidence mean ≥ 0.75
- 至少一个非泛化 process_term

规则 4：泛化模块过滤

以下类标签默认降级：

- cell stress
- apoptosis
- inflammation
- metabolism
- signaling abnormality

除非文本明确把它定义为 disease-central convergence mechanism。

5.4 causal chain 聚合规则

合格 chain 条件

- 至少 3 步
- steps 是有序事件，不是散点词
- 至少能落到某个模块
- 终点最好指向 phenotype / disease vulnerability / module exit

合并条件

两个 chain 可合并，当：

- module 相同
- 关键事件重叠 > 60%
- 起点与终点语义一致

过滤条件

丢弃：

- 只有两步的模糊链
- 纯描述性链，没有因果方向
- 明显只是一般 pathway 常识，而非 disease-relevant chain

5.5 key gene 聚合规则

core_driver

满足：

- 多来源反复出现
- 直接嵌入某核心模块
- 与 canonical chain 起点或关键节点相关

supporting_gene

满足：

- 仅在模块内有辅助作用
- 支持度有限
- 可能是扩图时保留但非 backbone 核心

5.6 支持度 scoring

给每个 hallmark/module/chain 计算 support_score。

示例公式

```
support_score =
  0.45 * source_diversity_score +
  0.25 * authoritative_source_score +
  0.20 * mean_llm_confidence +
  0.10 * disease_specificity_score
```

各项建议

- source_diversity_score: 不同 source_type 越多越高
- authoritative_source_score:
  - GeneReviews: 1.0
  - Orphanet: 0.9
  - ReviewArticle: 0.8
  - ReactomeSummary: 0.7
- disease_specificity_score:
  - 明确 disease-central: 1.0
  - 相关但非中心: 0.6
  - 泛化: 0.2

六、backbone 生成流水线设计

6.1 Stage 0：Disease initialization

输入：

```
{
  "disease": {
    "label": "Parkinson disease",
    "mondo_id": "MONDO:0005180"
  },
  "seed_genes": ["PRKN", "PINK1", "LRRK2", "GBA", "SNCA", "VPS35"]
}
```

输出：

- DiseaseDescriptor
- run_id
- workspace dirs

6.2 Stage 1：Source collection

收集来源建议按优先级：

优先层

- GeneReviews
- Orphanet
- 权威综述

补充层

- Reactome disease-relevant pathways
- GO summary terms
- ClinGen / OMIM summary

输出：

- raw source docs
- source manifest

6.3 Stage 2：Source packetization

把长文档切成 packet，建议按：

- section
- subsection
- disease mechanism paragraph group

切分，不建议机械按 token 块切。

输出：

- `source_packets.jsonl`

每个 packet 都保留：

- source type
- 标题
- section label
- 原文 text
- disease id

6.4 Stage 3：LLM constrained extraction

对每个 SourcePacket 调 Prompt A，产出 ExtractionResult。

输出：

- `extraction_results.jsonl`

同时记录：

- llm raw response
- parse status
- schema validation status

6.5 Stage 4： Normalization

程序做标准化：

hallmarks

- normalize label
- 去停用词
- 归并常见同义短语

modules

- 映射 canonical module label
- process_terms 规范化

genes

- 映射 HGNC symbol
- 去重

relations

- predicate 映射到 controlled vocab

输出：

- normalized candidates

6.6 Stage 5： Aggregation

对所有 normalized candidates 聚合：

- hallmark clusters
- module clusters
- chain clusters
- gene-role clusters

输出：

- `aggregation_records.json`

6.7 Stage 6：Scoring and pruning

对聚合结果做 support scoring 和裁剪：

保留为 core

- support_score ≥ threshold
- disease specificity 足够
- 机制边界明确

保留为 provisional

- 有潜力但支持不足

丢弃

- 太泛
- 低支持
- 和 disease 关联弱

输出：

- scored items
- prune log

6.8 Stage 7：Backbone assembly

组装成最终 draft：

- hallmarks
- modules
- module_relations
- canonical_chains
- key_genes
- build quality summary

输出：

- `disease_backbone_draft.json`

6.9 Stage 8：Validation

验证项：

- hallmark 数量是否合理
- module 数量是否合理
- 每个 core module 是否有 source support
- 每个 chain 是否至少 3 步
- 每个 key gene 是否链接到模块

- 是否存在过泛模块漏过滤

输出：

- `validation_report.json`

6.10 Stage 9：Review package export

给人工快速审阅用：

- backbone draft
- 每个模块的 supporting snippets
- 待审 flags
- 低置信项列表

输出：

- `review_bundle/`

七、目录结构建议

```
disease_backbone_builder/
├─ app/
│  ├─ schemas/
│  │  ├─ builder_config.py
│  │  ├─ disease_descriptor.py
│  │  ├─ source_packet.py
│  │  ├─ extraction_result.py
│  │  ├─ candidates.py
│  │  ├─ aggregation.py
│  │  ├─ backbone_draft.py
│  │  └─ validation_report.py
│  ├─ prompts/
│  │  ├─ packet_extraction.txt
│  │  ├─ module_refinement.txt
│  │  └─ merge_assist.txt
│  ├─ services/
│  │  ├─ source_collector.py
│  │  ├─ packetizer.py
│  │  ├─ llm_extractor.py
│  │  ├─ normalizer.py
│  │  ├─ aggregator.py
│  │  ├─ scorer.py
│  │  ├─ pruner.py
│  │  ├─ assembler.py
│  │  └─ validator.py
│  ├─ pipelines/
│  │  └─ build_backbone.py
```

```
│   └─ utils/
│       ├─ text_normalize.py
│       ├─ ontology_mapper.py
│       └─ json_io.py
├─ data/
│   ├─ raw_sources/
│   ├─ source_packets/
│   ├─ extraction_results/
│   ├─ aggregation/
│   ├─ outputs/
│   └─ review_bundle/
└─ tests/
```

八、最小可用版本的规则配置

建议先用一个可维护的 yaml/json 配置。

```yaml
hallmark:
  min_core_support_count: 2
  max_count: 8

module:
  min_core_support_count: 2
  max_count: 12
  generic_filter_terms:
    - stress
    - apoptosis
    - inflammation
    - metabolism
    - signaling abnormality

chain:
  min_steps: 3
  min_confidence: 0.7

gene:
  min_confidence_for_core_driver: 0.8

source_weights:
  GeneReviews: 1.0
  Orphanet: 0.9
  ReviewArticle: 0.8
  ReactomeSummary: 0.7
  GOSummary: 0.6
```

九、builder 的关键工程约束

9.1 不直接生成最终真值

输出永远叫：

- draft
- candidate
- provisional

9.2 每个对象都要挂 source

否则不可追溯。

9.3 先 disease-level，再 gene-level

Builder 做 backbone，不做全量扩图。

9.4 优先可审阅，而不是一步到位

因为 backbone 的价值在于：

- 可复核
- 可扩展
- 可修订

十、你在 PD 上的预期输出长什么样

最后产物应该接近这样：

```
{
  "backbone_id": "pd_backbone_draft_v1",
  "disease": {
    "label": "Parkinson disease",
    "ids": {
      "mondo": "MONDO:0005180",
      "mesh": "D010300"
    }
  },
  "hallmarks": [
    {
      "label": "mitochondrial dysfunction",
      "support_score": 0.94,
      "source_count": 3,
      "status": "core"
    },
    {
```

```
      "label": "impaired mitophagy and autophagy",
      "support_score": 0.92,
      "source_count": 3,
      "status": "core"
    }
  ],
  "modules": [
    {
      "label": "mitophagy",
      "description": "Selective clearance of damaged mitochondria.",
      "key_genes": ["PINK1", "PRKN", "FBXO7"],
      "support_score": 0.93,
      "status": "core"
    }
  ],
  "canonical_chains": [
    {
      "title": "PINK1-PRKN failure impairs damaged mitochondria
clearance",
      "module_label": "mitophagy",
      "support_score": 0.9,
      "status": "core"
    }
  ],
  "key_genes": [
    {
      "symbol": "PRKN",
      "gene_role": "core_driver",
      "linked_modules": ["mitophagy"]
    }
  ]
}
```

十一、你下一步最应该做的事

先不要急着做所有疾病。

先做一个 **PD backbone builder MVP**：

1. 选 1 个 GeneReviews disease section
2. 选 1 个 Orphanet summary
3. 选 2 到 3 篇高质量综述
4. 跑 packet extraction
5. 做 aggregation
6. 输出 `pd_backbone_draft_v1.json`

只要这个闭环跑通，你的 builder 就成型了。

十二、一句话收束

这套 Disease Backbone Builder 的本质是：

> **用 LLM 把权威疾病资料转成“可追溯的结构化 backbone 候选”，再用程序做标准化、聚合、裁剪和验证，最终得到一个可审阅、可扩展的 disease backbone draft。**